NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD

Parallel and Distributed Computing (LAB)

Lab Report: 03

Submitted to
Sir Farhad Riaz

Submitted By
Junaid Asif
(BSAI-144)

Submission Date: October 20, 2025

Task 1

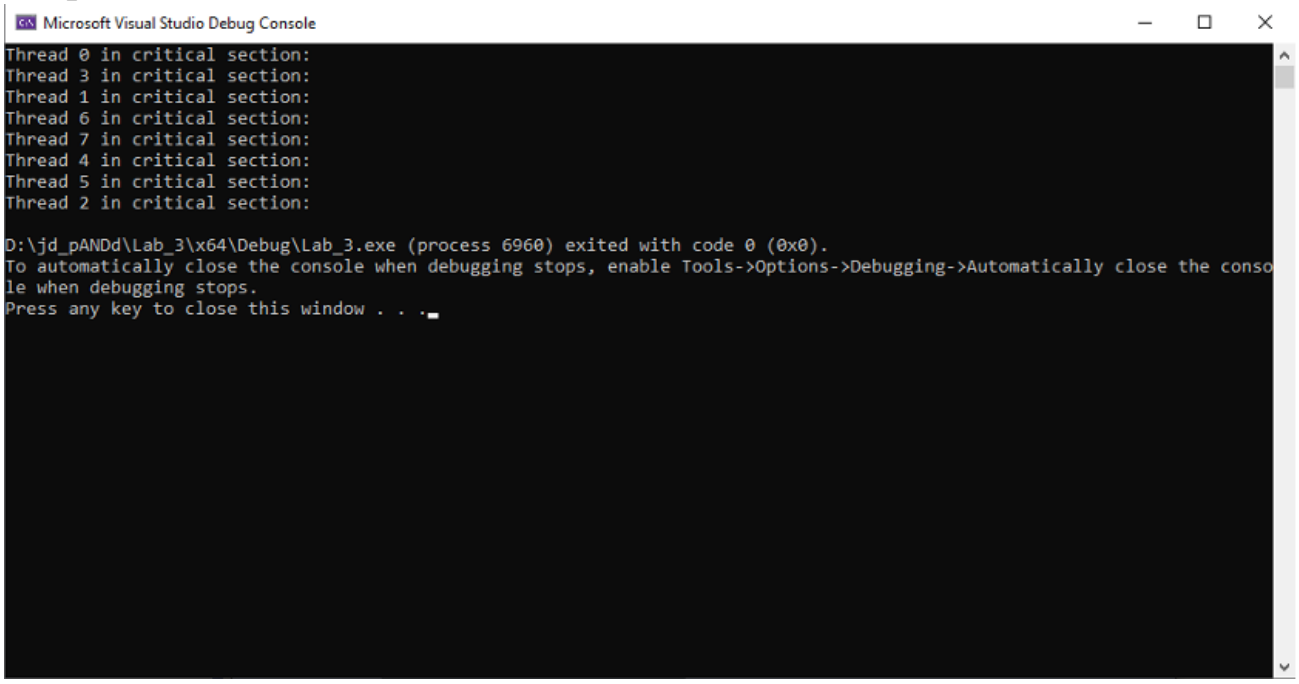
Problem Statement

Write a program to demonstrate the use of the #pragma omp critical directive for creating a thread-safe section.

Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
   #pragma omp parallel
   {
      #pragma omp critical
      {
         printf("Thread %d in critical section:\n", omp_get_thread_num());
      }
   }
}
```

Output



Explanation

- #pragma omp critical ensures **only one thread at a time** executes the enclosed code block.
- It prevents **race conditions** and maintains **data consistency**.
- Here, each thread prints its ID sequentially instead of overlapping output.

Why we use it

Because when multiple threads access shared resources (like console I/O), simultaneous access can produce **corrupted output** or **inconsistent results**.
The critical directive enforces mutual exclusion.

Task 2

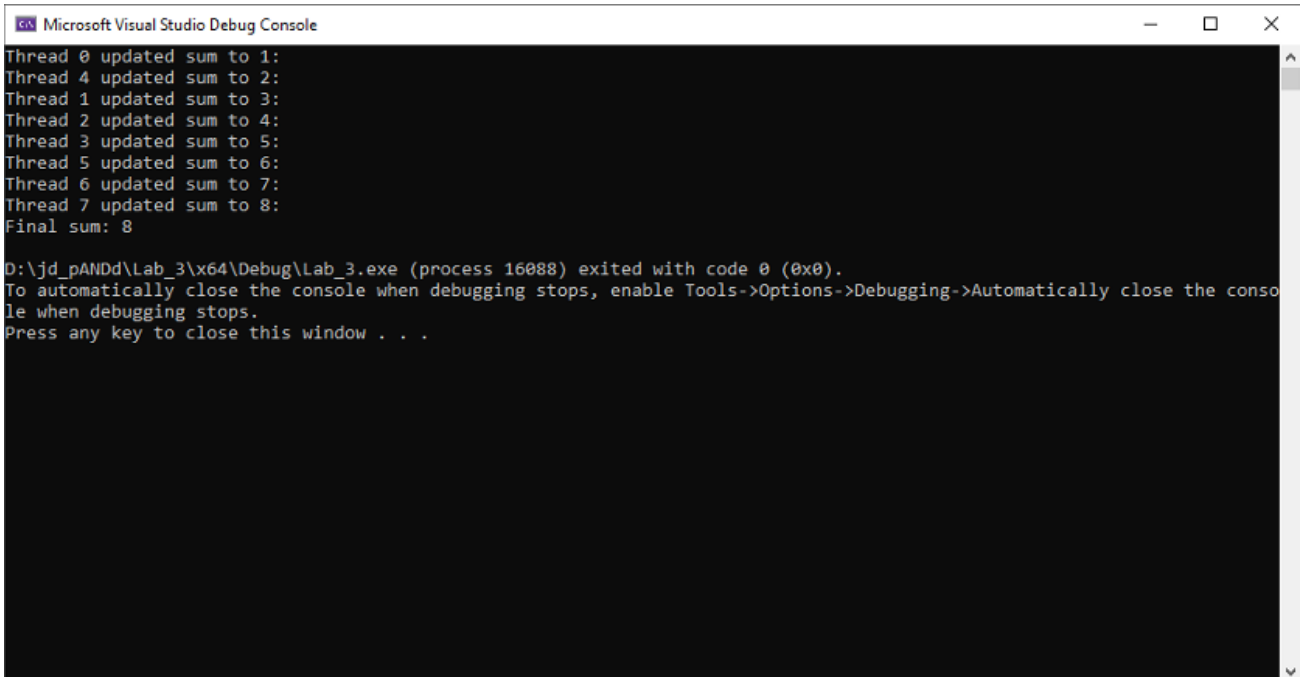
Problem Statement

Use a critical section to safely increment a shared variable (sum) across multiple threads.

Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
   int sum = 0;
   #pragma omp parallel
   {
      int thread_id = omp_get_thread_num();
      #pragma omp critical
      {
         sum += 1;
         printf("Thread %d updated sum to %d:\n", thread_id, sum);
      }
   }
   printf("Final sum: %d\n", sum);
   return 0;
}
```

Output



Explanation

- Shared variable: sum
- Each thread increments sum **one at a time** due to the critical section.
- Ensures consistent updates and avoids race conditions.

Why we use it

Without synchronization, multiple threads would write to sum simultaneously, leading to **lost updates**.
critical guarantees **atomic updates** on shared variables.

Task 3

Problem Statement

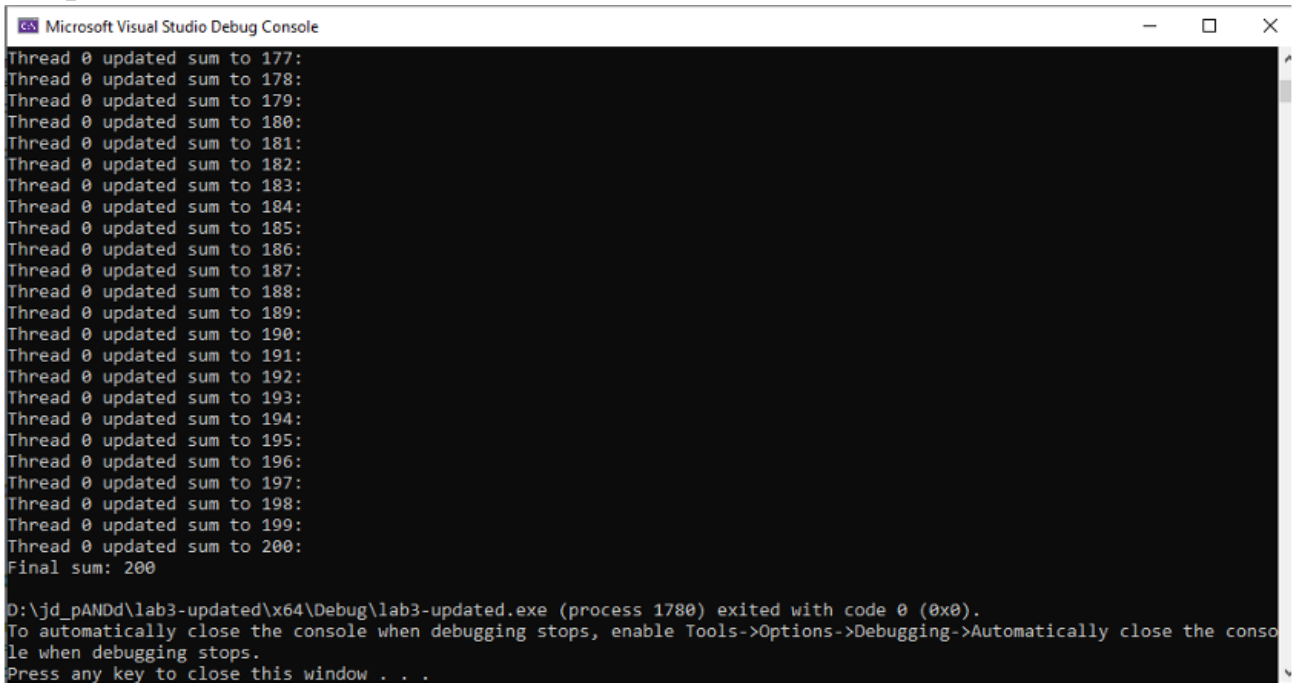
Demonstrate nested parallelism with a large number of threads updating a shared variable inside a critical section.

Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
```

```
    int sum = 0;
    int n = 200;
    #pragma omp parallel num_threads(40)
    {
        #pragma omp parallel for
        for (int i = 0; i < n; i++)
        {
            #pragma omp critical
            {
                sum += 1;
                printf("Thread %d updated sum to %d:\n", omp_get_thread_num(), sum);
            }
        }
    }
    printf("Final sum: %d\n", sum);
    return 0;
}
```

Output



Explanation

- This code uses **nested parallelism**, creating many threads.
- Each thread repeatedly updates sum under a critical section.

- However, too many threads competing for the critical section cause **overhead and reduced efficiency**.
-

Why we use it

It demonstrates the **performance trade-off**: while synchronization ensures correctness, too many synchronized threads can cause **bottlenecks**.

Task 4

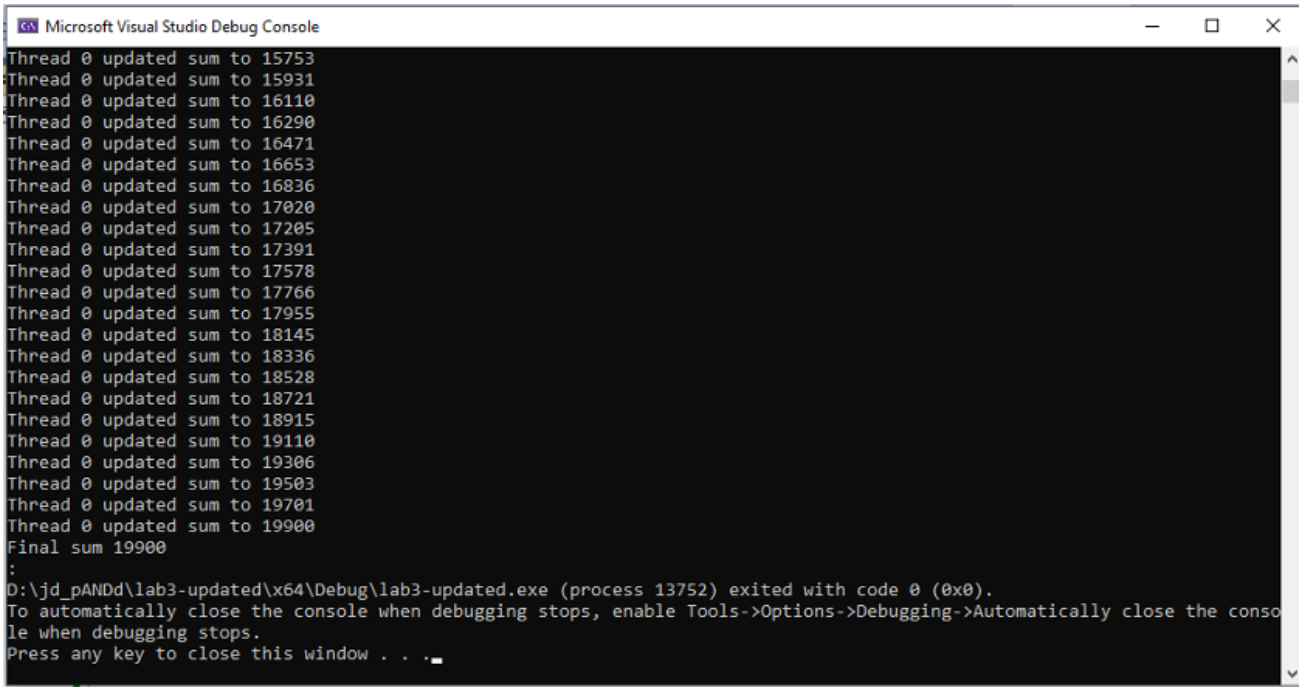
Problem Statement

Perform summation inside a critical section using multiple threads and loop iterations.

Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
   int sum = 0;
   int n = 200;
   #pragma omp parallel num_threads(40)
   {
      #pragma omp parallel for
      for (int i = 0; i < n; i++)
      {
         int thread_id = omp_get_thread_num();
         #pragma omp critical
         {
            sum += i;
            printf("Thread %d updated sum to %d\n", thread_id, sum);
         }
      }
   }
   printf("Final sum: %d\n", sum);
   return 0;
}
```

Output



Explanation

- Multiple threads calculate cumulative sums of integers.
- The critical region ensures sequential updates to the shared sum.
- Output may appear unordered but values remain consistent.

Why we use it

Demonstrates how critical prevents **data races** in accumulation operations involving shared variables.

Task 5

Problem Statement

Use #pragma omp parallel for with a critical region for safe updates and display both intermediate and final results.

Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
   int sum = 0;
```

```
    int n = 240;
    #pragma omp parallel for
    for (int i = 0; i < n; i++)
    {
        int thread_id = omp_get_thread_num();
        #pragma omp critical
        {
            sum += 1;
            printf("Thread %d updated sum to %d\n", thread_id, sum);
        }
    }
    printf("Final sum %d\n", sum);
    return 0;
}
```

Output

(Add screenshot here)

Explanation

- Parallel loop distributes iterations among threads.
- The critical region ensures one thread at a time updates the sum.
- Combines **parallel for** (work-sharing) with **critical** (synchronization).

Why we use it

Demonstrates **combined usage of parallelism + synchronization** for safe concurrent computation.

Task 6

Problem Statement

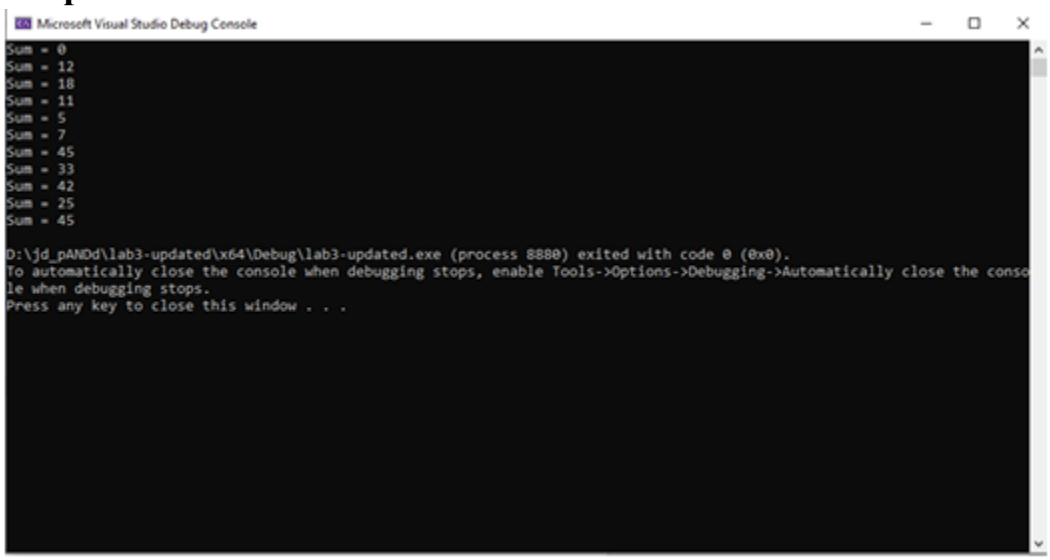
Use the #pragma omp atomic directive for lightweight synchronization.

Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
    int sum = 0;
```

```
    #pragma omp parallel for
    for (int i = 0; i < 10; i++)
    {
        #pragma omp atomic
        sum += i;
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

Output



Explanation

- atomic performs synchronization **only for a single memory operation**.
- It's faster than critical because it locks only a **small portion** of code.
- All threads safely update sum concurrently with minimal contention.

Why we use it

When synchronization is required on **simple operations** like addition, atomic is preferred for **efficiency**.

Task 7

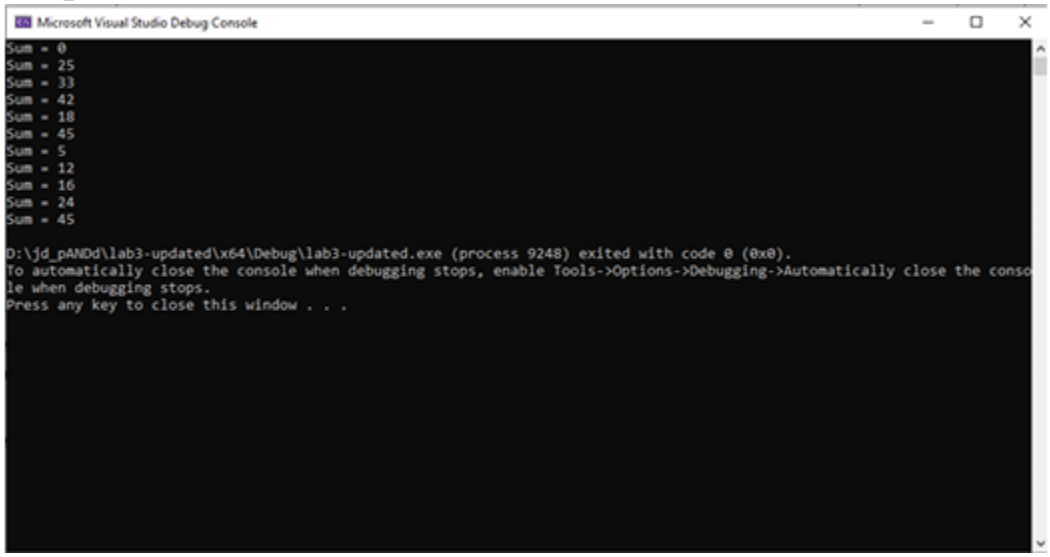
Problem Statement

Compare critical with atomic by using critical to update the sum variable.

Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
   int sum = 0;
   #pragma omp parallel for
   for (int i = 0; i < 10; i++)
   {
      #pragma omp critical
      sum += i;
   }
   printf("Sum = %d\n", sum);
   return 0;
}
```

Output



Explanation

- Each addition is performed inside a critical region.
- Works correctly but slower than using atomic due to **full section locking**.
- Both yield the same result, but with different performance characteristics.

Why we use it

To compare **critical vs atomic** — correctness is same, but atomic is faster and

preferred for simple memory updates.

Task 8

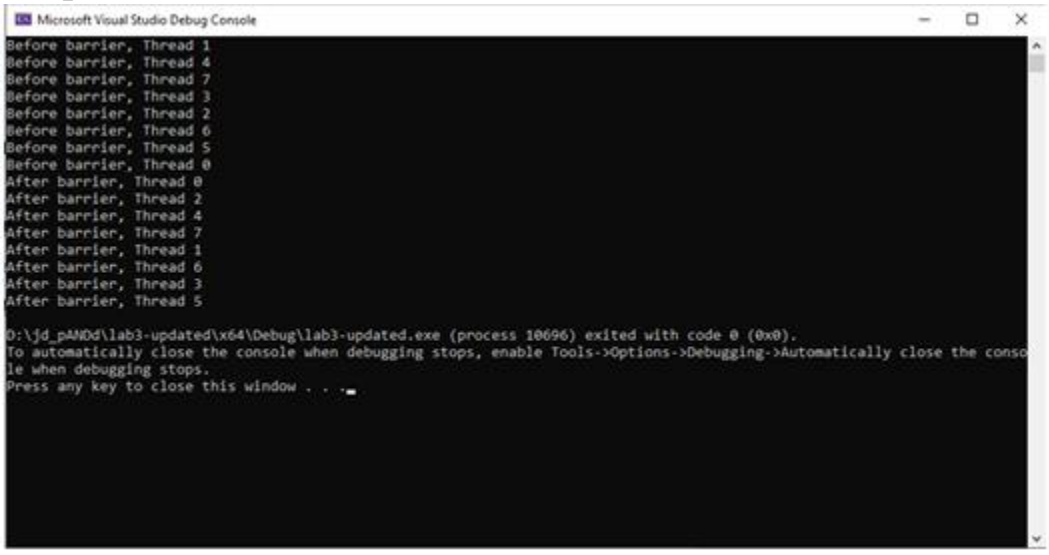
Problem Statement

Demonstrate use of #pragma omp barrier to synchronize all threads before proceeding.

Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
   #pragma omp parallel
   {
      printf("Before barrier, Thread %d\n", omp_get_thread_num());
      #pragma omp barrier
      printf("After barrier, Thread %d\n", omp_get_thread_num());
   }
   return 0;
}
```

Output



Explanation

- All threads execute until the barrier, then **wait for others** to reach it.
- After all threads synchronize, execution continues.
- Useful for **synchronizing phases of parallel computation**.

Why we use it
Barriers are essential to **coordinate parallel tasks**, ensuring all threads finish one step before starting another.

Task 9

Problem Statement
Use barriers to synchronize shared array updates among threads.

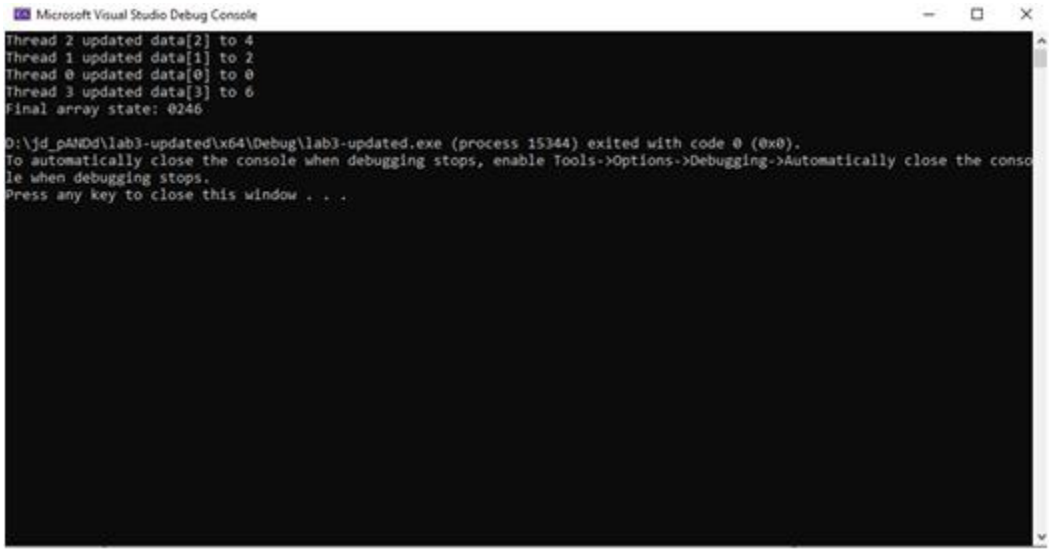
Code

```
#include <stdio.h>
#include <omp.h>
int main()
{
   int n = 4;
   int data[4] = {0, 0, 0, 0};
   #pragma omp parallel num_threads(n)
   {
      int thread_id = omp_get_thread_num();
      data[thread_id] = thread_id * 2;
      printf("Thread %d updated data[%d] to %d\n", thread_id, thread_id,
data[thread_id]);

      #pragma omp barrier

      if (thread_id == 0)
      {
         printf("Final array state: ");
         for (int i = 0; i < n; i++)
            printf("%d ", data[i]);
         printf("\n");
      }
   }
   return 0;
}
```

Output



Explanation

- Each thread updates a unique array index.
- Barrier ensures **all updates complete** before printing.
- Only thread 0 prints the final state of the array after synchronization.

Why we use it

Demonstrates **coordination between threads** modifying shared data and safely consolidating results.

Comparative Analysis of Synchronization Techniques

Directive	Purpose	Effect	When to Use	Performance Impact
critical	Ensures mutual exclusion	Only one thread executes code at a time	When multiple operations or print statements need protection	Slower (heavier lock)
atomic	Synchronizes single memory operations	Reduces locking overhead	For simple operations like sum += i	Faster (lightweight)
barrier	Synchronizes thread execution points	Forces all threads to wait	When all threads must reach same point before continuing	Moderate

Directive	Purpose	Effect	When to Use	Performance Impact
parallel for	Distributes loop iterations	Speeds up computation	For independent iterations	Efficient, scalable
num_threads()	Controls number of threads	Defines workload distribution	To control load balancing	Depends on core count

Overall Learning Summary

1. **Synchronization is vital** for data integrity in shared-memory parallelism.
2. critical is versatile but slower; atomic is specialized and faster.
3. barrier ensures all threads are coordinated before moving ahead.
4. Proper synchronization balances **correctness and performance**.
5. Overusing synchronization (especially critical) can **reduce parallel efficiency**.

Conclusion

This lab successfully demonstrated OpenMP synchronization directives — critical, atomic, and barrier.
By understanding their effects on performance and correctness, we learn how to write **safe, efficient, and scalable** parallel programs.
Choosing the right directive depends on the **operation complexity** and the **degree of shared data interaction** among threads.